Assessment 5 — Full Project Documentation

Project title: Metabolic Risk Digital Twin (NHANES 2017–2018)

1) Project overview

In this assessment, we built a lightweight “patient digital twin” framework to predict metabolic risk using the NHANES 2017–2018 dataset. The goal was to keep the model interpretable and reproducible, and then operationalize it through a simple, usable demo stack:

- A reproducible training pipeline (scripts + versioned artifacts)
- A FastAPI backend for inference and what-if analysis
- A React (Vite) dashboard for interactive inputs, model switching (L2/L1), and scenario simulation

This document explains the dataset setup, modeling decisions, system architecture, how to run the project, and how we verified it.

2) Quick start (local)

2.1 Python environment

```
".venv/bin/python" -m pip install -r requirements.txt
".venv/bin/python" -m pip install -e .
```

2.2 Run the API

```
".venv/bin/python" -m uvicorn healthcare_digital_twin.api:app --reload --port 8000
```

2.3 Run the UI

```
cd frontend
npm install
npm run dev
```

Open: `http://127.0.0.1:5173`.

3) Repository structure

- `data/raw/` — NHANES XPT files (source)
- `data/processed/` — cleaned patient dataset used for modeling
- `src/healthcare_digital_twin/` — reusable Python package (data, modeling, inference, API)
- `scripts/` — CLI entrypoints (`build_dataset.py` , `train.py` , `predict.py`)
- `artifacts/` — trained pipelines + metadata JSON (reproducibility evidence)
- `reports/figures/` — evaluation figures saved at training time
- `frontend/` — React UI (Vite) that calls the API

4) Dataset and label

Dataset: NHANES 2017–2018.

Integration approach: We merge multiple NHANES tables using the participant identifier `SEQN` to create one patient-level row per participant.

Processed dataset: `data/processed/patient_state_clean.csv`.

Target label: `Metabolic_Risk` is created from HbA1c thresholding (consistent with our research pipeline). HbA1c is used only to build the label and is not part of the input feature set.

5) Feature set (Digital Twin inputs)

The deployed system uses 9 features (recorded in artifact metadata for traceability):

- `Age`
- `Gender` (0=Male, 1=Female)
- `BMI`
- `Systolic_BP`
- `Diastolic_BP`
- `Glucose`
- `Insulin`
- `Total_Cholesterol`
- `HDL_Cholesterol`

6) Modeling approach

6.1 Pipeline

Training and inference use a single scikit-learn Pipeline:

1. Median imputation
2. Standard scaling
3. Logistic Regression classifier

Using the same pipeline at train and serve time helps avoid train/serve mismatch and keeps the preprocessing consistent.

6.2 Train/test split and tuning

- Stratified train/test split (`test_size=0.2` , `random_state=42`)
- Stratified 5-fold cross-validation for tuning
- Primary optimization metric: F1

6.3 Regularization modes (L2 vs L1)

We support two regularization settings through separate saved artifacts:

- **L2 (default):** stable baseline performance
- **L1 (sparse):** encourages sparsity and can simplify driver reporting (useful for interpretability)

7) Training + artifacts (CLI)

7.1 Build the processed dataset (optional)

```
".venv/bin/python" scripts/build_dataset.py
```

7.2 Train + export the L2 artifact

```
".venv/bin/python" scripts/train.py --model logreg --tune --threshold 0.4 --
artifact-name logreg_pipeline
```

7.3 Train + export the L1 artifact

```
".venv/bin/python" scripts/train.py --model logreg --tune --penalty l1 --threshold
0.4 --artifact-name logreg_pipeline_l1
```

7.4 Exported outputs

Artifacts (examples):

- `artifacts/logreg_pipeline.joblib`
- `artifacts/logreg_pipeline.metadata.json`
- `artifacts/logreg_pipeline_default.joblib`
- `artifacts/logreg_pipeline_default.metadata.json`
- `artifacts/logreg_pipeline_l1.joblib`
- `artifacts/logreg_pipeline_l1.metadata.json`

Figures (examples):

- `reports/figures/logreg_pipeline.confusion_matrix.png`
- `reports/figures/logreg_pipeline_l1.confusion_matrix.png` (if produced)

8) Evaluation results (from saved artifacts)

All metrics are stored in the artifact metadata JSON to keep our results reproducible.

- **Default decision rule (no manual threshold):** F1 ≈ 0.6844 (`logreg_pipeline_default.metadata.json`)
- **Screening threshold (threshold = 0.4):** F1 ≈ 0.7090 (`logreg_pipeline.metadata.json`)

Design note: For this project, we focus on clinically meaningful and user-actionable views (probability, thresholding behavior, confusion matrix, and what-if changes). ROC-style plots are intentionally not used in the UI.

9) Backend API (FastAPI)

Start the backend:

```
".venv/bin/python" -m uvicorn healthcare_digital_twin.api:app --reload --port 8000
```

9.1 Endpoints

- `GET /health` → service readiness
- `POST /predict` → risk probability + label

- `GET /model-info` → available models, artifact metadata, dataset overview (dashboard)
- `POST /analyze` → baseline + scenarios with delta probability and top drivers (what-if)
- `GET /figures/{name}` → serves PNG figures from `reports/figures/` (hardened)

9.2 `/predict` contract

Request body: a `PredictionRequest` containing the 9 features.

Optional query parameters:

- `model=l2|l1` (select artifact alias)
- `artifact_name=<name>` (direct override)
- `threshold=<float>` (override decision threshold)

Example:

```
curl -s "http://127.0.0.1:8000/predict?model=l2" \
    -H "Content-Type: application/json" \
    -d
'{"Age":45,"Gender":1,"BMI":28.2,"Systolic_BP":130,"Diastolic_BP":82,"Glucose":110,"Ins
```

9.3 `/analyze` contract (what-if)

`/analyze` accepts a baseline patient and a list of scenario overrides. It returns:

- Baseline probability + drivers
- Scenario probability + delta vs baseline + drivers

This endpoint is what the UI uses to generate the probability-vs-Δ curve.

9.4 CORS / frontend compatibility

Local development CORS is configured for common Vite origins (for example, ports `517x`).

10) Frontend (React dashboard)

Start the UI:

```
cd frontend
npm install
npm run dev
```

Open: `http://127.0.0.1:5173`.

Optional backend URL override:

- Copy `frontend/.env.example` → `frontend/.env.local`
- Set `VITE_API_BASE_URL=http://127.0.0.1:8000`

10.1 Implemented UI features

- Input form with validation (aligned to API schemas)
- Model selector: L2 (default) vs L1 (sparse)
- Prediction output: probability + threshold + risk label
- What-if simulator:

    - user changes one feature by Δ
    - UI calls `/analyze` and shows the probability change
    - UI renders a **dynamic curve** (probability vs Δ) that changes based on the baseline inputs
    - UI shows top drivers for the selected scenario
- Dashboard:
    - dataset overview (rows, positive rate, female rate)
    - model metrics (from metadata)
    - confusion matrix image served via `/figures/{name}`

11) Testing and quality gates

11.1 Backend automated tests

```
.venv/bin/python -m pytest -q
```

Latest executed result: **9 passed**.

11.2 Frontend gates

```
cd frontend
npm run lint
npm run build
```

Both pass in the current environment.

11.3 Reports

- Test report: `docs/assessment5/TEST_REPORT.md` and `docs/assessment5/TEST_REPORT.pdf`
- Validation report: `docs/assessment5/VALIDATION_REPORT.md` and `docs/assessment5/VALIDATION_REPORT.pdf`

12) Demo flow (presentation checklist)

1. Start the backend and show `GET /health`
2. Open the UI and confirm the API connected status
3. Enter patient inputs and run a prediction (probability + label)
4. Switch model (L2 ↔ L1) and show that the dashboard updates
5. Run a what-if scenario (example: increase `Glucose`) and show the Δ probability + curve update
6. Show the confusion matrix as the fixed test-set evaluation (it does not change per patient)

13) Limitations

- NHANES is cross-sectional; this is a statistical risk estimator intended for academic/demo use.
- The system is not validated for clinical decision-making.
- External generalization requires evaluation on local populations.

14) Troubleshooting

- If `uvicorn` fails to start:
    - install dependencies: `".venv/bin/python" -m pip install -r requirements.txt`

    - ensure editable install: `".venv/bin/python" -m pip install -e .`
    - if port 8000 is busy, use `--port 8001`
- If the UI shows “API disconnected”:
    - confirm the backend is running
    - confirm `VITE_API_BASE_URL` matches the backend URL
    - CORS is preconfigured for local dev origins; custom origins must be explicitly allowed

15) Appendix: PDF export commands

```
npx --yes md-to-pdf docs/assessment5/ASSESSMENT_5_DOCUMENTATION.md
npx --yes md-to-pdf docs/assessment5/TEST_REPORT.md
npx --yes md-to-pdf docs/assessment5/VALIDATION_REPORT.md
```